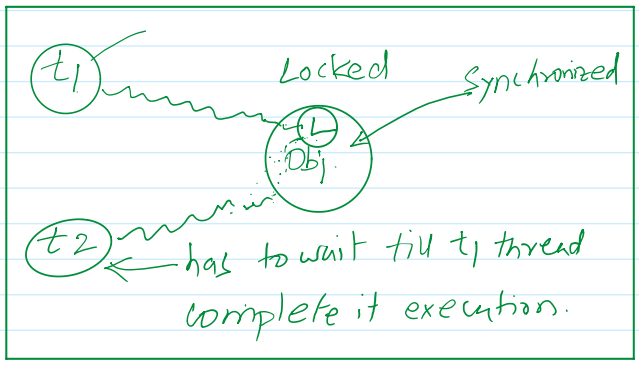
Deadlock

06 January 2026 10:17

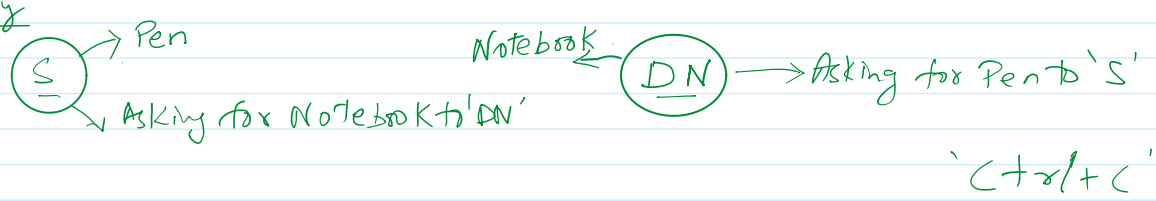
If two threads are waiting for each other forever, such type of infinite waiting is called Deadlock.



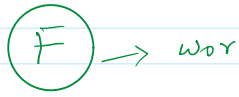
"synchronized" keyword is the only reason for deadlock situation, hence by using "synchronized" keyword we have to take special care. There is no resolution technique for deadlock. But prevention techniques are available.

Prevention technique:- never call another synchronized method from a synchronized method, if the case is happening, revise the logic of program design.

Real life Analogy



'S' & 'DN' both wait forever
'This is the deadlock'



(Deadlock situation).

Conditions for Deadlock.

1. Mutual Exclusion → Resource can be used by only one thread at a time.
2. Hold and Wait → Thread holds one resource and waits for another.
3. No Preemption → Resource cannot be forcibly taken.
4. Circular wait → Threads wait in a circular chain.

```
package MyThreads.Deadlocks;
public class DeadLockOne {
    static final Object resource1 = new Object();
    static final Object resource2 = new Object();
    public static void main(String[] args) {
        Thread t1 = new Thread(()->{
```

```
            synchronized(resource1){
                System.out.println("Thread 1: locked resource1");
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized(resource2){
                    System.out.println("Thread 1: locked resource2");
                }
            }
        });
        Thread t2 = new Thread(()->{
            synchronized(resource2){
                System.out.println("Thread 2: locked resource2");
                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized(resource1){
                    System.out.println("Thread 1: locked resource1");
                }
            }
        });
        t1.start();
        t2.start();
    }

}
```

How Deadlock occurs in this Program.

1. Thread1 locks resource1.
2. Thread2 locks resource2.
3. Thread1 waits for resource2.
4. Thread2 waits for resource1.
5. Both waits forever → Deadlock.

Detecting Deadlock

Java provides ThreadMXBean for detecting deadlocks:

```
package MyThreads.Deadlocks;
public class DeadLockOne implements Runnable{
    final Object resource1;
    final Object resource2;

    public DeadLockOne(Object lock1, Object lock2) {
        this.resource1 = lock1;
        this.resource2 = lock2;
    }
    @Override
    public void run() {
```

```java
        synchronized(resource1){
            System.out.println(Thread.currentThread().getName() + " acquired lock1");
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            synchronized(resource2){
                System.out.println(Thread.currentThread().getName() +
                " acquired lock2");
            }
        }
        synchronized(resource2){
            System.out.println(Thread.currentThread().getName() + " acquired lock1");
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            synchronized(resource1){
                System.out.println(Thread.currentThread().getName() +
                " acquired lock2");
            }
        }
    }
}
```

```java
package MyThreads.Deadlocks;
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;
public class DeadLockDetector {
    public static void detectDeadLock() {
        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] ids = bean.findDeadlockedThreads();
        if(ids != null){
            System.out.println("Deadlock detected!");
            ThreadInfo[] infos =
                bean.getThreadInfo(ids, true, true);

            for(ThreadInfo info : infos){
                System.out.println("Thread name: " + info.getThreadName());
                System.out.println("Thread state: " + info.getThreadState());
                System.out.println("Locked on : " +info.getLockName());
                System.out.println("Owned by: " + info.getLockOwnerName());
                System.out.println("Stack Trace:");
                for(StackTraceElement ste : info.getStackTrace()){
                    System.out.println("\t" + ste);
                }
                System.out.println("----------------------------------------");
            }
        }
        else{
            System.out.println("No deadlock detected.");
        }
    }
}
```

Scans JVM, finds deadlocked thread IDs

gets stack trace
shows locked resource
shows owning thread

```java
package MyThreads.Deadlocks;
public class DeadlockTest {
    public static void main(String[] args){
        Object r1 = new Object();
        Object r2 = new Object();
        Thread t1 = new Thread(new DeadLockOne(r1, r2), "Thread-1");

        Thread t2 = new Thread(new DeadLockOne(r1, r2), "Thread-2");
        t1.start();
        t2.start();
        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
        }

        DeadLockDetector.detectDeadLock();
    }
}
```

Two threads locks resource in opposite order.

Both wait for each other's lock

```
Output:

Thread-1 acquired lock1
Thread-1 acquired lock2
Thread-2 acquired lock1
Thread-1 acquired lock1
Deadlock detected!
Thread name: Thread-1
Thread state: BLOCKED
Locked on : java.lang.Object@7cd84586
Owned by: Thread-2
Stack Trace:
        app//MyThreads.Deadlocks.DeadLockOne.run(DeadLockOne.java:37)
        java.base@21.0.1/java.lang.Thread.runWith(Thread.java:1596)
        java.base@21.0.1/java.lang.Thread.run(Thread.java:1583)
----------------------------------------
Thread name: Thread-2
Thread state: BLOCKED
Locked on : java.lang.Object@1ddc4ec2
Owned by: Thread-1
Stack Trace:
        app//MyThreads.Deadlocks.DeadLockOne.run(DeadLockOne.java:23)
        java.base@21.0.1/java.lang.Thread.runWith(Thread.java:1596)
        java.base@21.0.1/java.lang.Thread.run(Thread.java:1583)
----------------------------------------
```

Important Notes:

Works for:

- synchronized locks
- Reenterant lock
- Ownable synchronizers

Return

- null -> No deadlock
- long[] -> Thread IDs involved.

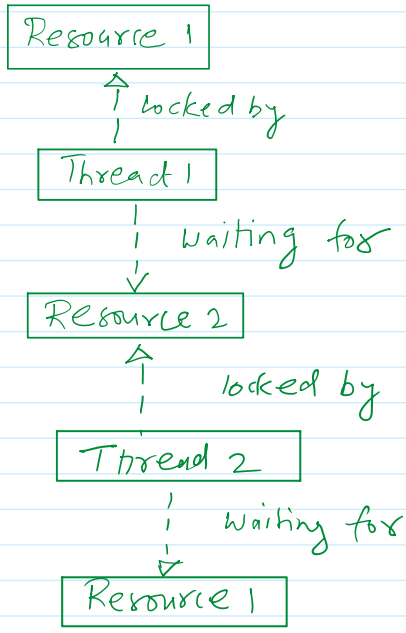
Summary:

ThreadMXBean provides a programmatic way to:

- Detect deadlocks

- Inspect involved threads
- Get stack traces
- Analyze root cause

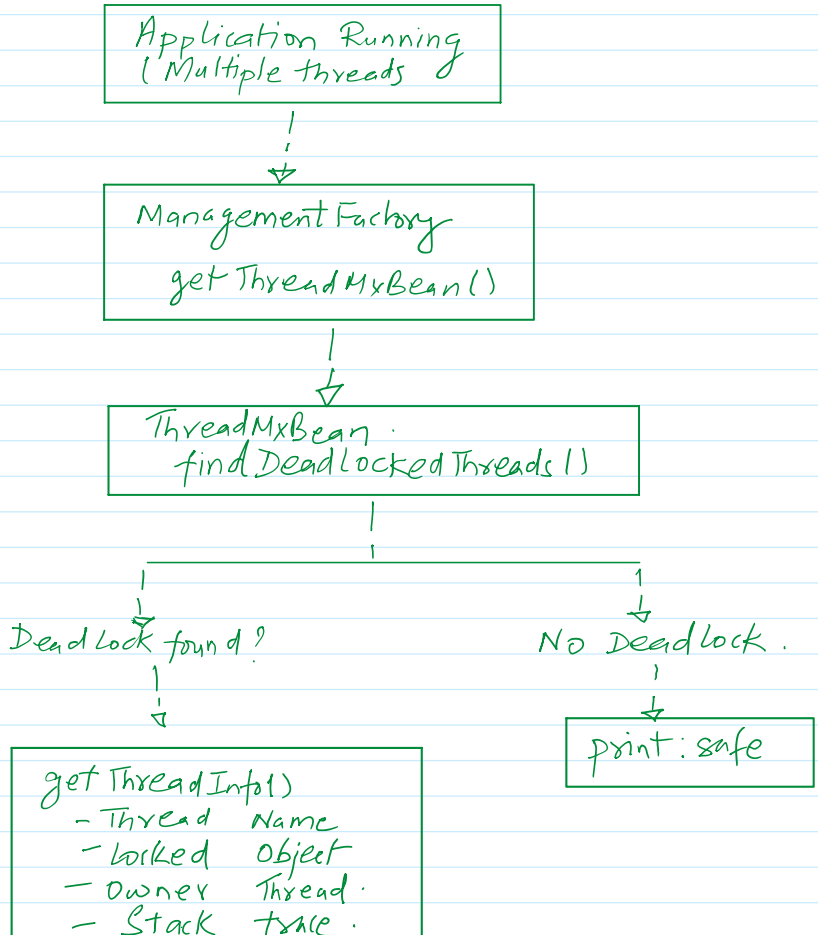
This is a production grade debugging techniques used in real JVM monitoring system.



Thread t1 → Resource 2
Resource 2 → Thread t2
Thread t2 → Resource 1
Resource 1 → Thread 1

No thread can proceed.
Circular dependency formed.
Program freezes.

Deadlock Detection using ThreadMxBean – Flow diagram.



- Locked Object
- Owner Thread.
- Stack trace.

Log/ Alert Admin